Meeting 16
Dimensionality, PCA, and PageRank

October 26, 2026 · Understanding the shape of data · Graphs acting on data

A table can contain many features even when most of its variation follows only a few directions. Finding those directions helps us understand the data and can make it cheaper to store and process. A graph offers a different kind of structure: relationships between entities tell us how to combine their values. We develop both ideas: first through PCA and the singular value decomposition, then through PageRank.

The main points

1. **The number of recorded coordinates can exceed the number needed to describe the data.** A cloud with three coordinates can lie in a plane. Sections 1–2.
2. **The SVD identifies perpendicular directions and their spreads.** Zero singular values indicate exact dependencies; small ones suggest possible approximations. Sections 2–3.
3. **Removing a direction means subtracting a projection.** Choosing the direction with least variance minimizes the average squared movement. Section 4.
4. **Variance accounts for what is kept and what is discarded.** Fewer directions can save storage and computation, with an approximation error we can measure. Sections 5–6.
5. **PageRank repeatedly redistributes a total score of one.** Senders divide their scores among outgoing links; receivers add the incoming shares. Restarts allow score to reach every node. Sections 7–8.
6. **Message passing is a matrix-vector product.** The graph supplies the weights, and the current scores supply the data. Sparse storage makes this practical for large networks. Sections 9–10.

1. How many dimensions does the data really use?

Begin with points described by two coordinates. A round cloud spreads substantially in both directions. Squeeze the cloud until it is very thin, and it begins to look like a line. Each point still has two recorded coordinates, but one coordinate along the line may describe it well.

The **ambient dimension** is the number of feature columns. The dimension of the linear structure occupied by the centered data can be smaller. If the points lie exactly on a line, one coordinate along that line suffices. If they lie near it, a one-dimensional representation is an approximation.

2. From a flat cloud to the SVD

In two dimensions, a correlation close to $+1$ or -1 is a clue that two nonconstant features nearly satisfy a linear relation. With more features, checking individual pairs is not enough. A dependency may involve three or more columns together.

The rotating three-dimensional example made this visible. From one angle the cloud looked three-dimensional; from another it looked flat. The relation in the demonstration was

$$x_1 + x_2 + 1.5x_3 = 0\,.$$

Thus $x_3 = -(x_1 + x_2) / 1.5$. The third coordinate is determined by the other two, even though all three coordinates vary and no pair needs correlation ± 1. The cloud spans a plane and has rank 2.

Think of a thin disk of stars in three-dimensional space. Looking at it face-on reveals its width; looking edge-on reveals how little thickness it has. Rotating our viewpoint does not change the stars. It reveals structure that was harder to see from the first angle.

In many dimensions we cannot rotate a picture and inspect it. **Principal component analysis (PCA)** describes variation using perpendicular directions chosen from the data. The **singular value decomposition (SVD)** supplies these directions. For PCA, first subtract the mean of each feature column. Write the resulting centered matrix as X, with n observations and m features:

$$X = USV^T\,.$$

For the tall-matrix case used in the explanation, $n \geq m$, the reduced factors have these shapes:

Object	Shape	Meaning
X	$n \times m$	Centered data
U	$n \times m$	One row per observation; coefficients used in reconstruction
S	$m \times m$	Diagonal matrix of singular values
V	$m \times m$	Perpendicular unit directions in feature space

The dimensions match: $(n \times m)(m \times m)(m \times m)$ produces an $n \times m$ matrix. The singular values are nonnegative and conventionally ordered $\sigma_1 \geq \cdots \geq \sigma_m \geq 0$.

> **Centering and correlation.**
>
> Centering subtracts column means. Standardizing also rescales columns, usually by their standard deviations. These are different operations. Also, the full correlation matrix can reveal dependencies; the limitation above concerns inspecting its pairwise entries for values near ± 1.

3. Directions, spread, and approximate dimension

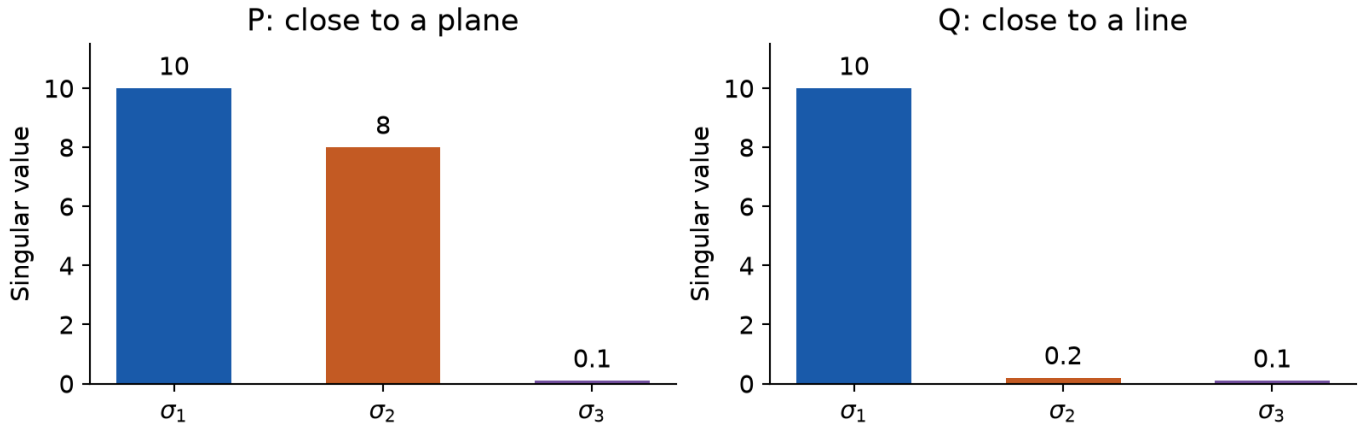
In a two-dimensional cloud, the first principal direction follows the greatest spread. The second is perpendicular to it. In three dimensions there are three mutually perpendicular directions. These directions depend on the data: they need not coincide with the original coordinate axes.

The directions in V have unit length. The arrows drawn over the cloud were scaled to show spread. For a centered dataset, using the population convention with divisor n, the spread along principal direction v_j is

$$\textit{standard deviation}_j = \frac{\sigma_j}{\sqrt{n}}, \qquad \text{variance}_j = \frac{\sigma_j^2}{n}.$$

When the cloud becomes flatter, the singular value associated with its thickness decreases. If that value reaches zero, the cloud lies exactly in a lower-dimensional subspace. If it is merely small, the exact rank may remain unchanged, but a lower-dimensional approximation may be useful.

The **rank** is the number of nonzero singular values. Approximate dimension asks a different question: how many directions must we keep to retain enough of the variation for our purpose?



Both clouds have condition number 100, but their useful approximations have different dimensions.

The condition number compares the largest and smallest singular values:

$$\kappa = \frac{\sigma_1}{\sigma_m}.$$

A large ratio indicates very unequal spreads. It does not, by itself, tell us how many directions matter. In the examples above, $(10, 8, 0.1)$ is close to a plane, while $(10, 0.2, 0.1)$ is close to a line. Both have $\kappa = 100$. We need to inspect the values between the largest and smallest as well.

4. Find a small direction, then remove it

To understand what the decomposition is looking for, choose any unit direction w. For a centered point x, treated here as a column vector, its signed coordinate along that direction is $x^T w$. The projected vector is

$$\text{proj}_w(x) = (x^T w)w.$$

Project every point onto the line. Because the data are centered, the projected coordinates have mean zero. Their variance measures the spread along w:

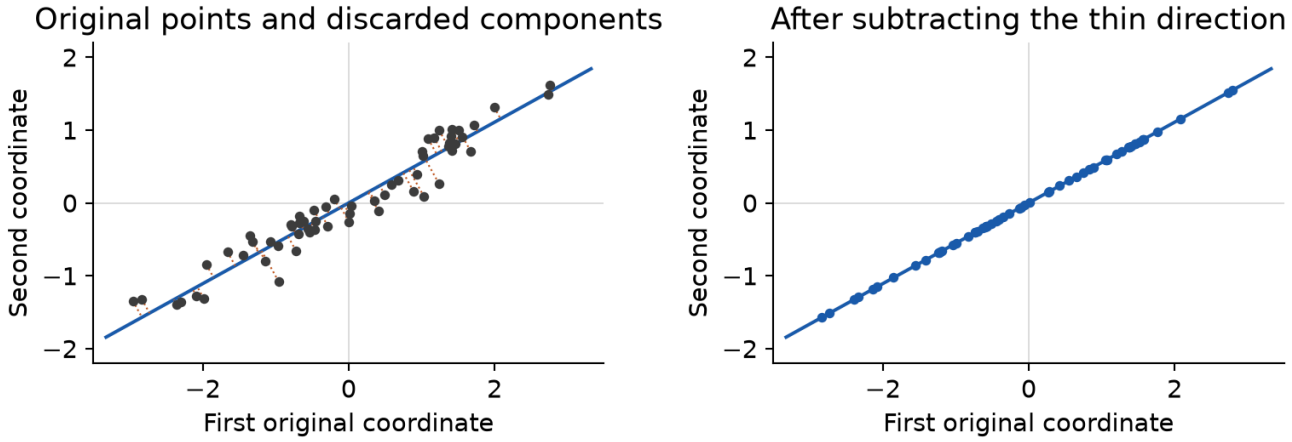
$$\text{var}\,(w) = \frac{1}{n}\sum_{i=1}^{n}(x_i^T w)^2 = \frac{1}{n}\|Xw\|^2, \qquad \|w\| = 1\,.$$

Turning the line changes the projections and their variance. For a thin three-dimensional cloud, the direction perpendicular to its nearly planar structure has small variance: most points project close to the center along that direction.

Once we find this least-variance direction, remove its component from each point:

$$x_{\text{new}} = x - (x^T w)w\,.$$

The result lies in the plane perpendicular to w. Keeping the projection onto w would leave a line; subtracting it leaves the complementary plane. Each point moves by its discarded component, so the average squared movement equals the variance along w.



The same idea in two dimensions: removing the thin direction leaves a line. In three dimensions, it leaves a plane. Illustrative points.

Now repeat *inside the remaining plane*. Find its least-variance direction, remove that component, and keep the perpendicular line within the plane. In m dimensions, each step reduces the dimension by one. The directions found along the way are mutually perpendicular.

> **The order of the construction.**
>
> The lecture built the picture by discarding the smallest-variance direction first. SVD notation lists singular values in the opposite order, largest first. The later directions removed in this construction have greater variance; removing dimensions does not increase the total variance that remains.

5. Account for all the variation

For centered data, the total variance is the average squared distance of the observations from the origin. Perpendicular coordinates split each point's squared distance into a sum of squared components. Adding this identity across all observations gives

$$\frac{1}{n}\sum_{i=1}^{n}\|x_i\|^2 = \sum_{j=1}^{m}\frac{\sigma_j^2}{n}.$$

This is the Pythagorean theorem applied to the new axes. It is a statement about **squared distances**, not a sum of ordinary distances. Equivalently, if $C = X^T X / n$ is the covariance matrix, total variance is the sum of its diagonal entries.

For a cloud of points spread across a floor, the two horizontal directions account for most of the variation. Measure deviations from the mean of the points and add squared distances to quantify that spread.

The variance share of principal direction j is

$$\frac{\sigma_j^2}{\sum_{\ell=1}^{m}\sigma_\ell^2}.$$

The shares sum to one when the dataset has nonzero total variance. This lets us quantify what a smaller representation retains instead of deciding only from the appearance of a plot.

Computing the decomposition

NumPy returns the singular values as an array and the right singular-vector matrix transposed. This reference code expands the lecture's function call.

Code: centered data, singular values, and variance

```
import numpy as np
# A: observations in rows, features in columns
mu = A.mean(axis=0)
X = A - mu
U, s, Vt = np.linalg.svd(X, full_matrices=False)
n = X.shape[0]
variance = s**2 / n
total_variance = variance.sum()
if total_variance > 0:
    variance_share = variance / total_variance
```

`full_matrices=False` requests the reduced SVD. With $r = \min(n, m)$, the shapes are $U{:}\,n \times r$, $s{:}\,r$, and $V^T{:}\,r \times m$. This avoids unnecessary full-size factors; it neither makes them sparse nor selects components to keep. See the NumPy SVD documentation.

For an exact dependency, a nonzero direction w satisfies $Xw = 0$. Its projected coordinates, variance, and singular value are zero: the algebraic version of the opening flat cloud.

6. Smaller representations and low-rank updates

Keeping the k directions with largest singular values gives a rank-at-most-k approximation:

$$X \approx X_k = U_k S_k V_k^T.$$

A point can then be represented with k coordinates rather than m. To reconstruct the original features, we also need the retained directions and the column means that were subtracted.

If we store the three factors separately, U_k has nk entries, the diagonal of S_k has k, and V_k^T has km. The count for the centered matrix is therefore

$$nk + k + km \quad \text{instead of} \quad nm\,.$$

Lecture example	Stored values
Full matrix: 50,000 rows × 300 columns	15,000,000
Factors retaining 20 directions	1,000,000 + 20 + 6,000 = 1,006,020

This is about fifteen times fewer values. Storing the 300 means adds a small extra cost if we want the original, uncentered data. Savings depend on choosing k small enough; a factorization using every component need not save storage.

Computation can also use the factors directly. For a vector z, evaluate $U_k(S_k(V_k^T z))$ from the inside out. This avoids rebuilding the full matrix.

> **Compression and noise.**
>
> Discarding only zero components preserves the data exactly, apart from numerical rounding. Discarding nonzero components is lossy. Small-variance directions may contain measurement noise, so removing them can improve a representation, but small variance alone does not establish that a direction is unimportant.

LoRA and fine-tuning

The meeting began with fine-tuning: adapting an already trained language model to more specialized material, such as company documents. The connection is that a useful weight update may be describable with far fewer parameters than a full weight matrix.

LoRA means **low-rank adaptation**. Instead of training every entry of a weight update independently, write

$$W_{\text{new}} = W + \Delta W, \qquad \Delta W = BA\,.$$

For the slide's 4,096 × 4,096 matrix, an unrestricted update has 16,777,216 entries. With B of shape 4,096 × 8 and A of shape 8 × 4,096, the two factors contain 65,536 trainable entries, 256 times fewer. Their product still has the full matrix shape but rank at most 8.

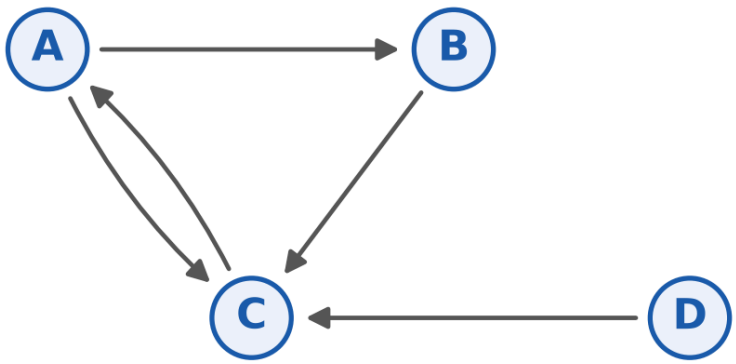
LoRA keeps the pretrained weights fixed and trains the smaller factors. The connection to this lecture is the economy of a low-rank representation; standard LoRA does not require computing the SVD of the pretrained matrix. See Hu and colleagues' LoRA paper.

7. PageRank and the web as a graph

The second part of the meeting turned from the geometry of a data table to relationships between entities. A web page is a node; a hyperlink is a directed edge. If page A links to page B, the edge points from A to B. The question is how to assign importance scores using this link structure.

PageRank was part of the early Google project developed by Larry Page and Sergey Brin at Stanford. Its central idea is recursive: a page receives importance from pages that link to it, and their importance depends on other links in turn. This supplies information beyond whether a page contains the words in a query. The NSF account of Google's origins describes the research setting.

The lecture used a graph with four pages and five links:



A sends to B and C. B and D send to C. C sends to A.

Give the graph a total score of one and distribute it equally at the start:

$$r^{(0)} = (0.25,\ 0.25,\ 0.25,\ 0.25)^T .$$

The lecture called this score **attention**. Each node holds a share, and the algorithm repeatedly moves those shares along links. Another interpretation is a population of walkers: a node's score is the fraction of the population at that node.

As with earlier matrix computations, it helps to separate the objects. The graph tells us where values can travel. The score vector tells us how much each node currently has.

8. Split, send, add, and restart

A sender divides its score equally among its outgoing links. A has score 0.25 and two outgoing links, so it sends $0.25 / 2 = 0.125$ to B and 0.125 to C. B has just one outgoing link, so its entire 0.25 goes to C. D does the same. C sends its entire score to A.

A receiver **adds** the shares that arrive. Thus C's score from the first round of link-following is

$$0.125 + 0.25 + 0.25 = 0.625 .$$

C does not divide this sum by its three incoming links. The division already occurred at each sender and used that sender's number of outgoing links.

All messages in a round use the old scores. After every receiver's sum is computed, the new vector replaces the old one:

$$(0.25,\ 0.25,\ 0.25,\ 0.25)^T \mapsto (0.25,\ 0.125,\ 0.625,\ 0)^T .$$

The total is still one. D receives zero because no page links to D. Repeating the operation makes scores depend on longer paths through the network.

Why add teleportation?

Link-following alone can confine attention to a closed group of pages. On some graphs it can also cycle instead of settling. Having no incoming links explains D's zero score; it does not, by itself, imply that the entire iteration fails to converge.

Teleportation, also called a restart, adds a chance of moving to any node. With the lecture's choice $\alpha = 0.85$, 85% of the attention follows links and 15% is distributed uniformly across all four pages. Each page receives a restart share of

$$\frac{1 - 0.85}{4} = 0.0375.$$

For C, the full update is $0.85(0.625) + 0.0375 = 0.56875$.

Page	Link-following sum	Score after restart is included
A	0.25	0.25
B	0.125	0.14375
C	0.625	0.56875
D	0	0.0375

Every node now receives some attention, and the total remains one. The choice 0.85 is a parameter, not a mathematical necessity. At $\alpha = 0$, all attention comes from uniform restarts, so all scores are equal. For $0 \le \alpha < 1$, with a valid link transition rule and uniform restarts, repeated updates approach a unique score vector.

In the demonstration, successive updates changed the scores less and less until they stabilized numerically. The resulting ranking for this graph at $\alpha = 0.85$ is C, A, B, D.

9. The sending rules become a matrix

Use the same node order, A, B, C, D, for every row, column, and score vector. The lecture's convention is

Column = sender. Row = receiver.

Start with a link matrix whose entry in row i, column j is one when j links to i. To obtain the sending matrix G, divide each sender's column by its number of outgoing links. For the example,

$$G = \begin{pmatrix} 0 & 0 & 1 & 0 \\ \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Column A is $(0, \frac{1}{2}, \frac{1}{2}, 0)^T$: A sends half to B and half to C. Column C is $(1, 0, 0, 0)^T$: C sends everything to A. Each column sums to one because a sender distributes its whole score.

Now read a row. Row C is $(\frac{1}{2}, 1, 0, 1)$, which describes the shares C receives from A, B, C, and D. Its dot product with the current score vector is

$$(Gr)_C = \frac{1}{2}r_A + r_B + 0r_C + r_D\,.$$

With the initial equal scores, this is exactly $\frac{1}{2}(0.25) + 0.25 + 0.25 = 0.625$. The dot product performs the same addition of messages that we described locally.

The full product Gr computes every receiver's sum at once. This is the lecture's **graph × data** viewpoint: the graph is represented by a matrix, and that matrix acts on values attached to the nodes.

The complete iteration

Let u be the uniform vector, with every entry $1 / N$ for a graph with N nodes. The PageRank update is

$$r^{(t+1)} = \alpha G r^{(t)} + (1 - \alpha)u\,.$$

The product computes incoming shares, the factor α keeps the link-following portion, and the final vector supplies restart shares. The result becomes the input for the next round.

> **Read the convention before using a matrix.**
>
> Some adjacency matrices use rows for senders. Here the senders are columns, which is why multiplying G by a column of scores gives the receiving totals. The graph is directed, so its matrix need not be symmetric.

The graph and the values play different roles, but both are data. Changing a link changes the sending rule; changing a score changes how much attention that rule distributes.

10. Computing PageRank efficiently

A loop over outgoing links and a matrix-vector product compute the same update. Both read the old scores and produce a separate new vector.

Code: one round by passing messages

```
import numpy as np
out = [[1, 2], [2], [0], [2]]  # A, B, C, D
old = np.full(4, 0.25)
alpha = 0.85
new = np.full(4, (1 - alpha) / 4)
for sender, receivers in enumerate(out):
    share = alpha * old[sender] / len(receivers)
    for receiver in receivers:
        new[receiver] += share
```

Once the sending fractions are stored in G, the update has the compact form shown in the lecture:

Code: the same round as a matrix product

```
G = np.array([[0,   0, 1, 0],
              [0.5, 0, 0, 0],
              [0.5, 1, 0, 1],
              [0,   0, 0, 0]])
u = np.full(4, 1 / 4)
new = alpha * (G @ old) + (1 - alpha) * u
# [0.25, 0.14375, 0.56875, 0.0375]
```

Here `@` means matrix multiplication. Each nonzero entry of G represents one weighted message.

> **The example's assumption.**
>
> Every node here has an outgoing link. A general implementation must also handle nodes without outgoing links, commonly by redistributing their score uniformly. D has an outgoing link but no incoming links.

A web graph has far fewer links than possible pairs of pages, so its matrix is **sparse**. Store existing links and their weights. With N nodes and E links, one round takes $O(E + N)$ work: process each link and account for the restart contribution at each node. Uniform restarts do not require a dense matrix of extra edges.

Graphs describe relationships. The lecture connected this with random walks and social networks: connections can reveal something about an entity before we inspect its individual features.

Sending values along links and adding them at receivers is matrix multiplication. PageRank repeats this operation with restarts to turn a network of relationships into a stable distribution of scores.